

Take-Home Assignment: AI-Powered Feedback Triage App

Context

Your team receives a steady stream of product feedback from users. Today this feedback is triaged manually, which is slow and inconsistent.

You will build a small, production-minded full-stack application that ingests feedback, stores it, and uses an AI model to generate structured analysis (summary, sentiment, tags, priority, and recommended next action).

You have complete freedom to choose your technology stack within the TypeScript/JavaScript ecosystem. The application should demonstrate clear separation of concerns:

- **Data Access:** Relational or non-relational database of your choice
- **Business Logic:** API endpoints and AI integration
- **Presentation:** Reactive UI for user interaction

Focus on modular, testable architecture with clear boundaries between concerns, component-oriented design, and pragmatic trade-offs you would make in production.

A note on AI tools

Use any AI tools you'd normally use day-to-day (Cursor, Copilot, Claude, etc.) — we're interested in understanding how you work. We'll ask why you chose a particular approach, what alternatives you considered, and how you'd modify or extend the code.

Requirements

Task 1: Backend API and Data Layer

Implement a server-side layer that exposes REST endpoints under `/api` to create and read feedback entries.

Endpoints

- `POST /api/feedback`
 - Accepts a JSON payload with at least `text` (string) and optional `email`.
 - The server must generate `createdAt`.
 - Returns the stored record including AI analysis fields.
- `GET /api/feedback`

- Returns a paginated list.
- Support server-side filters via query parameters: `sentiment` , `tag` .
- Include `page` and `pageSize` parameters with reasonable defaults.
- `GET /api/feedback/:id`
 - Returns a single record with full analysis.

Database

Choose any database solution you prefer. A single collection/table is sufficient.

Model (core fields):

- `id`
- `text`
- optional `email`
- `createdAt` (server-generated)

Analysis object:

- `summary` (string)
- `sentiment` (positive|neutral|negative)
- `tags` (string[])
- `priority` (P0–P3)
- `nextAction` (string)

API design and middleware

- JSON body parsing
- An `/api` router or equivalent routing structure
- Error-handling middleware returning structured errors
- Return `400` for invalid input and `5xx` for server errors with a consistent error shape
- Minimal request logging: method, path, status, latency, and a correlation/request ID

Configuration

- Environment-driven config (e.g., `DATABASE_URL` , `PORT`)
- Provide a `.env.example` with the expected variables

Optional (if time permits)

- Basic free-text search across `text` or `summary` . If added, document approach and trade-offs.

Task 2: AI Analysis Service (OpenAI or Anthropic)

Create a backend service module responsible for calling an AI API and returning a structured analysis for a given feedback text.

Provider

- Use either OpenAI's Node SDK or Anthropic's TypeScript SDK (choose one provider and explain the choice in SOLUTION.md).
- Authentication from an environment variable (e.g., `OPENAI_API_KEY` or `ANTHROPIC_API_KEY`).
- If you don't have a key, provide a mock mode that returns deterministic outputs.

Call pattern

- Perform the AI analysis synchronously within the `POST /api/feedback` request for this exercise.
- In SOLUTION.md, explain how you would evolve this to an async background process in production.

Prompting and output

- Instruct the model to return a strict JSON object with fields:
 - `summary`
 - `sentiment` (positive|neutral|negative)
 - `tags` (1–5 short nouns)
 - `priority` (P0–P3)
 - `nextAction`
- Parse/validate the model output; handle malformed responses gracefully.
- Include basic safety: keep outputs concise/professional and avoid echoing PII.

Performance and resilience (choose one to implement)

- Implement either:
 - Simple caching (e.g., hash the text to cache analysis), OR
 - Lightweight retries with backoff for transient failures
 - Log AI request metadata (timestamps, durations, model) but avoid logging raw user content or full prompts.
-

Task 3: Frontend Layer

Create a minimal frontend application that communicates with your backend.

Views and components

- **Submit Feedback page**
 - Form with a textarea for `text` and optional `email`.
 - On submit, call the API, show loading state, then success/failure feedback.

- **Feedback List page**
 - Paginated table/list showing `summary` , `sentiment` , `tags` , `priority` , and created date.
 - Server-side filters for `sentiment` and `tag` , with UI controls that update the list.
- **Feedback Detail view** (modal, drawer, or page)
 - Show the full `text` , `analysis` , and metadata.

Reusability

- Implement at least one reusable UI component (e.g., a Tag/Badge component used for `tags` and/or `priority`) to demonstrate scalable component design.

UX and configuration

- Use appropriate state management patterns for your chosen framework.
- Show meaningful empty, loading, and error states.
- Frontend reads backend base URL from an environment variable suitable for local development.

Task 4: Testing, Diagnostics, and Documentation

Demonstrate senior-level quality practices.

Tests

- **Backend:**
 - Unit test the AI analysis service by mocking the SDK client and verifying JSON parsing and error handling.
 - Include at least one additional test for a controller or data layer path (happy and error path recommended).
- **Frontend:**
 - Test one interactive component (e.g., the submit form or list filters) using accessible queries.

Diagnostics

- Structured logging on the server with a correlation/request ID.
- In `SOLUTION.md`, include a short "runbook" describing how you would debug:
 - AI rate limit or timeout errors
 - Database connection exhaustion or connectivity issues

Documentation (`SOLUTION.md` should include)

- Architecture overview (modules, data flow, components)
- Technology stack choices (framework, database, AI provider) and rationale
- Data model choices (relational vs non-relational) and rationale
- AI prompt engineering considerations and safety measures

- Which of caching or retries you implemented and why
 - Testing strategy and key scenarios
-

Deliverables

1. A single GitHub repository containing:
 - All the source code for the assignment
 - A README.md with instructions to run locally
 - A SOLUTION.md describing the design and trade-offs you made
 2. Video Demo (5–10 minutes using Loom or similar):
 - Demonstrate the working project by doing a voice over
 - Walk through your code architecture
 - Explain key design decisions and any trade-offs made
 - Highlight interesting technical implementations
-

Time Estimate

This should take ± 4 hours to complete, but you can take as long as you need.